

Facultatea de Electronică, Telecomunicații și Tehnologia
Informației

PROIECT RPI Guardian Dashboard

Student: Grecu Victor

Grupa: 433C

Prof. îndrumător: Voiculescu Valentin

București 2026

1. PREREQUISITES & RUN (Cerințe de sistem și rulare)

Pentru a putea rula această proiect, sistemul gazdă (Raspberry Pi 4) trebuie să aibă configurat un mediu virtual izolat, evitând astfel alterarea pachetelor globale din distribuția Linux (caracteristică obligatorie pe versiunile noi de Raspberry Pi OS bazate pe Debian Bookworm).

1.1. Structura necesară a fișierelor

Înainte de rulare, fișierele trebuie să fie organizate astfel:

```
tema_calp/  
├─ app.py  
├─ requirements.txt  
└─ templates/  
    └─ index.html
```

Atenție: fișierul `index.html` trebuie să se afle obligatoriu în subdirectorul `templates/`, altfel motorul de template-uri Jinja2 din Flask nu îl va găsi.

1.2. Pași pentru instalare și configurare

1. Se deschide un terminal pe Raspberry Pi, se creează directorul proiectului și se accesează acesta:

```
mkdir tema_calp  
cd tema_calp
```

2. Se creează folderul destinat interfeței grafice:

```
mkdir templates
```

3. Se creează mediul virtual Python (denumit `.venv`):

```
python3 -m venv .venv
```

4. Se activează mediul virtual:

```
source .venv/bin/activate
```

5. Cu mediul activat (fapt indicat de prefixul (.venv) în terminal), se instalează modulele necesare:

```
pip install flask
```

6. Pentru a genera fișierul de dependențe cerut la predare, se rulează comanda:

```
pip freeze > requirements.txt
```

1.3. Pornirea aplicației

Rularea temei se face prin pornirea scriptului principal Python din interiorul folderului rădăcină. Din terminalul în care mediul virtual este activat, se execută:

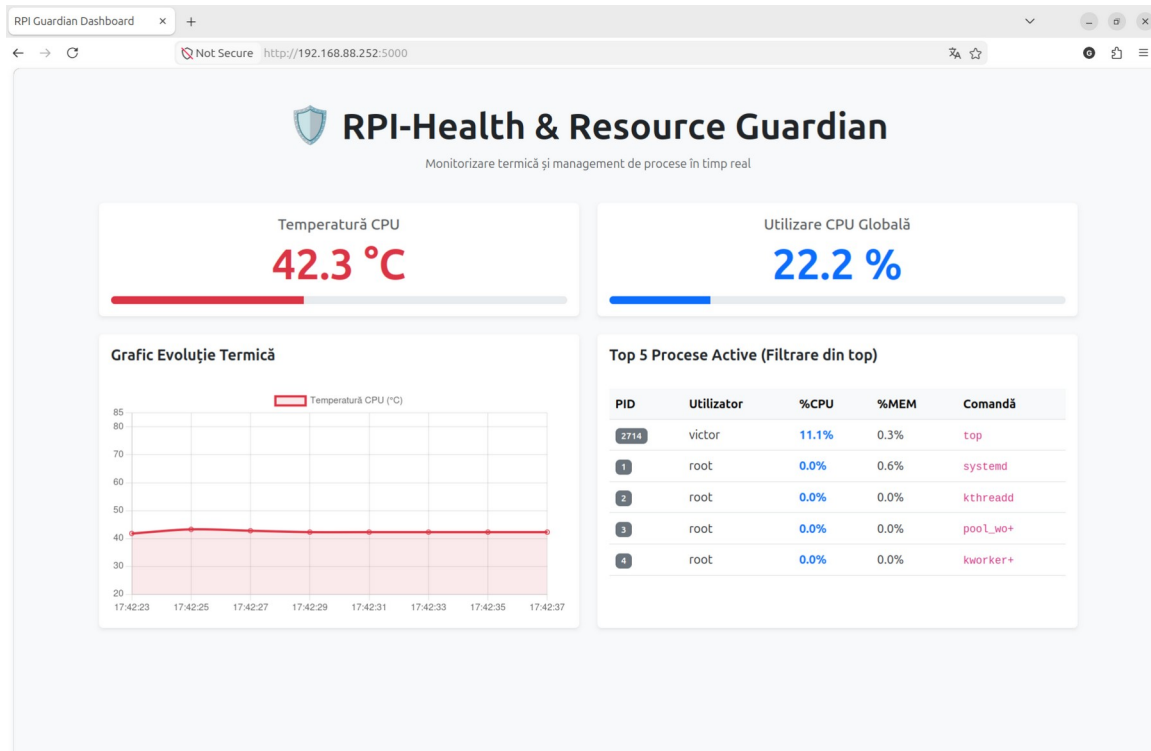
```
python3 app.py
```

După lansare, serverul Flask va începe să asculte pe toate interfețele de rețea disponibile pe portul 5000. Vizualizarea dashboard-ului se poate face local sau de pe un laptop aflat în aceeași rețea locală (sau hotspot), deschizând un browser web și accesând adresa:

```
http://<IP_RASPBERRY_PI>:5000    sau    http://localhost:5000
```

1.4. Screenshot-uri din timpul rulării

Mai jos sunt prezentate capturi de ecran din timpul rulării aplicației, realizate accesând dashboard-ul din browser:



Dashboard principal cu temperatura și utilizarea CPU afișate

```
(.venv) victor@raspberrypi:~/tema_calp $ python3 app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.88.252:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 855-251-635
192.168.88.238 - - [17/May/2026 17:42:22] "GET / HTTP/1.1" 200 -
192.168.88.238 - - [17/May/2026 17:42:23] "GET /favicon.ico HTTP/1.1" 404 -
192.168.88.238 - - [17/May/2026 17:42:23] "GET /api/stats HTTP/1.1" 200 -
192.168.88.238 - - [17/May/2026 17:42:25] "GET /api/stats HTTP/1.1" 200 -
192.168.88.238 - - [17/May/2026 17:42:27] "GET /api/stats HTTP/1.1" 200 -
192.168.88.238 - - [17/May/2026 17:42:29] "GET /api/stats HTTP/1.1" 200 -
192.168.88.238 - - [17/May/2026 17:42:31] "GET /api/stats HTTP/1.1" 200 -
192.168.88.238 - - [17/May/2026 17:42:33] "GET /api/stats HTTP/1.1" 200 -
192.168.88.238 - - [17/May/2026 17:42:35] "GET /api/stats HTTP/1.1" 200 -
192.168.88.238 - - [17/May/2026 17:42:37] "GET /api/stats HTTP/1.1" 200 -
192.168.88.238 - - [17/May/2026 17:42:39] "GET /api/stats HTTP/1.1" 200 -
192.168.88.238 - - [17/May/2026 17:42:41] "GET /api/stats HTTP/1.1" 200 -
```

Terminalul cu server-ul Flask pornit

```
~/tema_calp/system_history.csv - Mousepad
File Edit Search View Document Help
Timestamp, Temperature, CPU_Usage |
2026-05-17 17:42:23,41.8,21.4
2026-05-17 17:42:25,43.3,0.0
2026-05-17 17:42:27,42.8,25.0
2026-05-17 17:42:29,42.3,30.0
2026-05-17 17:42:31,42.3,30.0
2026-05-17 17:42:33,42.3,30.0
2026-05-17 17:42:35,42.3,22.2
2026-05-17 17:42:37,42.3,22.2
2026-05-17 17:42:39,42.8,25.0
2026-05-17 17:42:41,41.8,20.0
2026-05-17 17:42:44,42.3,25.0
2026-05-17 17:42:46,42.3,33.4
2026-05-17 17:42:48,44.3,33.4
2026-05-17 17:42:50,43.3,27.3
2026-05-17 17:42:52,43.3,30.0
2026-05-17 17:42:54,42.3,40.0
2026-05-17 17:42:56,42.3,0.0
2026-05-17 17:42:58,41.8,20.0
2026-05-17 17:43:00,41.3,25.0
2026-05-17 17:43:02,43.3,33.4
2026-05-17 17:43:04,42.3,25.0
```

Fișierul system_history.csv deschis

2. EXPLICAȚIE COD ȘI LOGICĂ DE IMPLEMENTARE

Aplicația a fost construită de la zero, combinând un backend în Python 3 capabil să invoce utilitare native din Linux CLI cu un frontend web ușor, bazat pe o pagină HTML generată dinamic prin intermediul motorului de template-uri Jinja2 (integrat în Flask).

2.1. Logica din Backend (app.py)

Fișierul app.py reprezintă nucleul aplicației. Acesta controlează atât achiziția de date din sistem, cât și expunerea acestora printr-un API de tip JSON.

În funcția `get_cpu_temperature()`, se apelează comanda CLI specifică platformelor Raspberry Pi: `vcgencmd measure_temp`. Aceasta returnează un string de tipul `"temp=48.2'C\n"`. Folosind metodele de manipulare de șiruri din Python (`.replace()`), eliminăm textul inutil și transformăm valoarea într-un număr real (float) pentru a putea fi prelucrată și trimisă ulterior către grafice.

La nivel global se verifică prezența fișierului system_history.csv. Dacă acesta lipsește, modulul os confirmă absența sa, iar modulul csv scrie prima linie

conținând capul de tabel. Funcția *save_log_to_csv(temp, cpu)* adaugă ulterior la finalul fișierului (mode='a'), la fiecare interogare, un rând nou format din timestamp-ul curent, valoarea temperaturii și procentul de utilizare CPU.

În funcția *get_top_processes()*, executăm comanda *top -b -n 1*. Flag-ul *-b* (batch mode) este critic deoarece oprește interfața interactivă a comenzii *top* din a rula la infinit, permițându-i să returneze un singur snapshot text complet în Python. Flag-ul *-n 1* limitează execuția la o singură iterație. Output-ul primit este un bloc masiv de text despărțit prin caractere de linie nouă (*\n*). Iterăm prin fiecare linie. Când analizăm linia globală de utilizare a procesorului (care conține string-ul "%Cpu(s)"), linia arată în format brut astfel:

```
%Cpu(s):  5.3 us,   2.1 sy,   0.0 ni, 92.6 id, ...
```

Pentru a extrage doar valorile numerice necesare, folosim instrucțiunea:

```
parts = line.split(' ', '')
```

Funcția *.split(' ', '')* rupe acea linie de text în mai multe bucăți, folosind virgula ca separator, și le salvează într-o listă numită *parts*. *parts[0]* va deveni string-ul: "%Cpu(s): 5.3 us", iar *parts[1]* va deveni string-ul: " 2.1 sy". Ulterior, pe fiecare dintre aceste sub-string-uri aplicăm din nou operații de tăiere prin spații goale și curățare de caractere (*.replace('%', '')*) pentru a izola valorile 5.3 (User space) și 2.1 (Kernel space/System). Adunându-le, obținem gradul real total de încărcare al procesorului: 7.4%.

Aceeași logică se aplică și mai jos în tabelul de procese, după ce trecem de linia cu header-ul "PID". Pentru fiecare linie ce descrie un proces activ, executăm *parts = line.split()*. Fără argumente, *.split()* taie linia după orice grup de spații libere. Astfel, *parts[0]* va fi întotdeauna coloana PID-ului, *parts[1]* utilizatorul, *parts[8]* consumul CPU, iar *parts[11:]* comanda completă. Acest mod de filtrare direct în memoria Python elimină riscul decalării coloanelor.

Pentru a evita reîncărcarea completă a paginii web la fiecare actualizare, am creat ruta */api/stats*. Aceasta apelează funcția *jsonify()*, transformând dicționarele și listele Python într-o structură nativă JSON ușor de citit de JavaScript.

2.2. Logica din Frontend (templates/index.html)

Interfața web folosește librăria Bootstrap 5 descărcată prin CDN pentru a genera un aspect curat, organizat sub formă de carduri de monitorizare și tabele responsive.

În secțiunea `<script>`, funcția asincronă *fetchStats()* interoghează endpoint-ul `/api/stats` la fiecare 2000 de milisecunde (*setInterval(fetchStats, 2000)*). Prin tehnologia modernă Fetch API, datele brute JSON sunt descărcate în fundal.

JavaScript preia valorile din JSON și modifică dinamic proprietățile text și barele de progres din HTML (*document.getElementById().innerText*). Pentru tabelul de procese, JavaScript golește corpul vechi al tabelului și generează rânduri noi folosind un șablon de tip text (template literal).

Biblioteca Chart.js desenează o linie temporală pe un obiect de tip `<canvas>`. La fiecare două secunde, noua valoare a temperaturii este adăugată în coada graficului, în timp ce valorile mai vechi de 20 de eșantioane sunt eliminate din listă prin apelul *.shift()*, menținând o fereastră glisantă dinamică.

3. REFERINȚE BIBLIOGRAFICE

În procesul de proiectare și implementare a acestei aplicații hibride Python-Linux, au fost utilizate următoarele surse și documentații tehnice oficiale:

1. Curs CALP – Prof. Valentin Voiculescu

URL: Platforma Moodle aferentă cursului

2. Ghid oficial Flask – Gestione rute și JSON

Documentația oficială pentru înțelegerea modului în care funcționează un micro-framework web și utilizarea metodei jsonify() pentru transmiterea datelor către componentele asincrone de JavaScript.

URL: <https://flask.palletsprojects.com/>

3. Documentație Python – Modul Subprocess

Suportul tehnic standard pentru executarea de comenzi Linux CLI din scripturi Python (check_output și run) și manipularea fluxurilor de ieșire (I/O streams).

URL: <https://docs.python.org/3/library/subprocess.html>

4. W3Schools – Python OS Module

Tutorial practic legat de manipularea căilor de fișiere din sistemul de operare Linux și verificarea existenței istoricului pe disc prin os.path.exists.

URL: https://www.w3schools.com/python/module_os.asp

5. Chart.js – Ghid pentru grafice liniare live

Resursă esențială utilizată la configurarea obiectului de tip chart în JavaScript, gestionarea culorilor canvas-ului și actualizarea asincronă a seturilor de date.

URL: <https://www.chartjs.org/docs/latest/charts/line.html>

6. Bootstrap 5 – Structuri de tabele și carduri responsive

Documentație de design folosită la așezarea în pagină a elementelor vizuale, stilizarea badge-urilor pentru PID-uri și generarea elementelor din clasa progress-bar.

URL: <https://getbootstrap.com/docs/5.3/content/tables/>

7. URL GitHub

URL: <https://github.com/IonGigi970/rpi-performance-monitor>